CÓDIGOCLARO · RUTA KOTLIN

Tema 13 · Coroutines

Material de estudio para DAM/DAW · Nivel progresivo · Edición 2026

Qué aprenderás

Este documento desarrolla el tema «Coroutines» dentro de la ruta de Kotlin. La meta no es memorizar una receta, sino entender los conceptos, escribir una solución legible, justificar decisiones y comprobar que el comportamiento coincide con el requisito.

Objetivos de aprendizaje

- Explicar con tus palabras los conceptos centrales de coroutines.
- Reconocer cuándo utilizar la técnica y qué alternativas existen.
- Leer y modificar el ejemplo guiado sin depender de copiarlo.
- Diseñar pruebas con un caso normal, un caso límite y un caso inválido cuando proceda.
- Localizar la documentación primaria adecuada para resolver dudas de versión.

Cómo estudiar este capítulo

Primero lee el modelo mental; después predice el resultado del ejemplo antes de ejecutarlo. Cierra el documento y reconstruye la solución desde cero. Por último, resuelve los ejercicios y contrasta tu razonamiento con los criterios orientativos. Marca una duda concreta para investigar en la documentación oficial.

Mapa del capítulo

1. Fundamentos · 2. Modelo mental · 3. Sintaxis · 4. Ejemplo comentado · 5. Diseño y límites · 6. Depuración · 7. Pruebas · 8. Prácticas · 9. Soluciones orientativas · 10. Repaso y fuentes.

1. Fundamentos

Una función suspend puede pausar y reanudarse mediante operaciones suspendibles como `delay`, sin bloquear el hilo mientras espera. La palabra clave no cambia por sí sola de hilo ni vuelve no bloqueante una llamada bloqueante; el dispatcher controla dónde se ejecuta la coroutine. `delay`, `launch` y `async` pertenecen a `kotlinx.coroutines`, que debe añadirse como dependencia. `launch` devuelve un Job; `async`, un Deferred con resultado; ambos se inician dentro de un CoroutineScope.

Idea esencial

El objetivo de «Coroutines» no es aprender una palabra aislada, sino comprender la relación entre entrada, transformación, estado y salida. En programación conviene describir el contrato antes de implementar: qué recibe la operación, qué garantiza, qué puede fallar y qué efectos secundarios produce.

Vocabulario que debes dominar

- Coroutines: nombre del concepto que organiza el capítulo.
- Contrato: condiciones de entrada y garantías de salida.
- Invariante: condición que debe seguir siendo cierta durante la ejecución.
- Caso límite: entrada situada en una frontera o condición poco habitual.
- Regresión: fallo que reaparece tras un cambio que parecía no relacionado.

Preguntas de comprobación

- ¿Qué problema resuelve coroutines?
- ¿Qué alternativa sencilla existe y qué coste tiene?
- ¿Qué dato haría que el enfoque fallara?
- ¿Cómo demostrarías que la solución es correcta?

2. Modelo mental y decisiones

Antes de escribir código, representa el proceso en tres pasos: entrada validada, transformación y resultado observable. Si existe estado mutable, identifica quién lo modifica y en qué momento. Si hay E/S o red, separa el cálculo de la operación externa para poder probar la lógica con datos controlados.

Descomposición del problema

- Entrada: qué valores necesita la solución de coroutines y quién los proporciona.
- Precondiciones: qué debe ser cierto antes de empezar.
- Proceso: qué regla o algoritmo transforma los datos.
- Salida: qué resultado devuelve o qué cambio observable realiza.
- Errores: cómo se representa un dato inválido o un fallo externo.
- Coste: si importa, estima el crecimiento de tiempo y memoria al aumentar los datos.

Decisión de diseño

Para coroutines, evita añadir abstracciones por adelantado. Empieza con la solución más pequeña que cumple el contrato, nombra los conceptos del dominio y separa responsabilidades cuando exista una razón comprobable: duplicación, complejidad, reutilización o aislamiento de efectos.

Ejercicio de razonamiento

Describe un caso real de DAM/DAW donde usarías coroutines. Escribe tres entradas, el resultado esperado de cada una y un ejemplo que deba rechazarse o tratarse de forma especial.

3. Sintaxis y patrones de uso

La sintaxis concreta depende de la tecnología y, en algunos casos, de la versión del compilador o del entorno. El patrón que conviene retener es el siguiente: expresar la intención directamente, mantener el alcance de las variables pequeño y evitar que una sola operación esconda decisiones distintas.

Ejemplo de referencia

```
import kotlinx.coroutines.delay
import kotlinx.coroutines.runBlocking

suspend fun cargar(): String {
    delay(100L)
    return "listo"
}

fun main() = runBlocking {
    // Solo como puente didáctico desde main: runBlocking bloquea este hilo.
    println(cargar())
}
```

Cómo leer el código

- Localiza la declaración o punto de entrada.
- Identifica el valor que se procesa y qué tipo tiene.
- Sigue la operación en orden; no asumas resultados sin evaluarlos.
- Localiza el resultado observable: retorno, salida, estado o escritura.
- Pregúntate qué ocurre si la entrada es vacía, nula o inesperada.

Variación guiada

Modifica el ejemplo para cubrir un segundo caso de uso de «Coroutines». No cambies varias cosas a la vez: introduce un cambio, predice su efecto, ejecútalo y anota si la predicción era correcta.

Nota de compatibilidad

Si un ejemplo no compila en tu entorno, registra el mensaje exacto y la versión del runtime. Comprueba primero las instrucciones oficiales; algunas APIs, opciones de sintaxis y herramientas de proyecto cambian entre versiones.

4. Ejemplo guiado paso a paso

Construye una solución pequeña relacionada con coroutines. El enunciado de práctica es: recibir datos de un caso sencillo, aplicar una regla clara y presentar el resultado de forma comprensible. Antes de programar, escribe un ejemplo de entrada y salida esperada.

Plan

- Reformula el enunciado sin palabras ambiguas.
- Elige nombres que expresen el significado, no solo el tipo.
- Implementa la regla central con el mínimo estado necesario.
- Añade una comprobación explícita de entrada si el contrato la requiere.
- Ejecuta el caso normal y compara el resultado con tu predicción.

Lectura crítica

El ejemplo de la página anterior ilustra un patrón frecuente en Kotlin, pero no es la única implementación correcta. Evalúa si la solución sigue siendo comprensible cuando el caso crece, si el error se comunica de manera útil y si las responsabilidades están separadas.

Registro de aprendizaje

Anota en tu cuaderno: qué parte entendiste a la primera, qué supuesto resultó falso y qué pregunta consultarás en la documentación. Esta pequeña bitácora facilita repasar antes de un examen y detectar patrones en tus errores.

Ampliación

Añade una segunda regla de negocio y mantén separada la lógica principal de la presentación. Si el ejemplo ya tiene demasiadas condiciones, divide la operación en funciones o componentes con nombres concretos.

5. Casos límite, errores y buenas prácticas

Una solución que funciona con un ejemplo no demuestra que funcione con todos los casos relevantes. Los errores más comunes suelen venir de supuestos no escritos: datos vacíos, valores fuera de rango, diferencias de mayúsculas, operaciones que fallan, estado desactualizado o dependencias de la versión.

Lista de comprobación

- Caso mínimo: ¿qué pasa con la entrada más pequeña válida para coroutines?
- Caso vacío: ¿se acepta, se devuelve un resultado vacío o se informa de un error?
- Caso límite: ¿qué ocurre justo en la frontera de una condición?
- Caso inválido: ¿se valida antes de usar el dato?
- Error externo: ¿qué ocurre si falla el archivo, la red o la dependencia?
- Legibilidad: ¿otra persona entiende el propósito sin conocer tu intención?
- Seguridad: ¿se evita ejecutar o interpretar contenido no confiable?

Buenas prácticas

Prefiere un comportamiento explícito a una conversión o mutación sorprendente. Evita capturar errores sin actuar, duplicar reglas de negocio y exponer información sensible en mensajes. Documenta el porqué de una decisión no obvia, no cada línea de código.

Criterio profesional

Valida las entradas, contempla los casos límite y comprueba el comportamiento con una prueba reproducible.

6. Depuración sistemática

Depurar no es cambiar líneas al azar: es reducir la distancia entre lo que esperabas y lo que observas. Reproduce el fallo con los pasos mínimos, guarda la entrada problemática y formula una hipótesis que pueda confirmarse o descartarse.

Procedimiento recomendado

- Reproduce el fallo de forma consistente.
- Lee el primer mensaje de error relevante y la traza completa.
- Inspecciona los valores justo antes de la operación que falla.
- Cambia una sola hipótesis por iteración.
- Añade una prueba de regresión cuando hayas corregido el defecto.
- Retira logs temporales que contengan datos sensibles y conserva los útiles.

Tabla de diagnóstico

Síntoma	Hipótesis	Comprobación
Resultado inesperado	La regla o entrada no era la esperada	Inspeccionar valores y repetir el caso
Falla solo en algunos datos	Caso límite o supuesto incorrecto	Reducir entrada y crear prueba específica
Falla en otra máquina	Versión, configuración o dependencia	Registrar runtime y reproducir instalación

Qué evitar

No declares resuelto un fallo porque desapareció una vez. Repite el escenario, prueba un caso vecino y verifica que la corrección no rompe un comportamiento anterior. Si no entiendes la causa raíz, la solución puede ser accidental.

7. Pruebas y criterios de calidad

Una prueba útil establece una condición inicial, ejecuta una acción y comprueba un resultado observable. Las pruebas no tienen que cubrir cada línea para ser valiosas; deben cubrir riesgos y contratos importantes. Separa las pruebas rápidas de unidad de las pruebas que requieren base de datos, navegador o red.

Matriz mínima de pruebas

Tipo	Qué valida	Ejemplo para este tema
Normal	Camino habitual	Entrada válida y resultado esperado
Límite	Frontera de una regla	Mínimo/máximo o colección vacía
Error	Fallo controlado	Dato inválido o dependencia fallida
Regresión	Corrección anterior	Repetir un caso que fallaba

Revisión de calidad

- Los nombres expresan intención y no inducen a error.
- Los errores tienen mensajes útiles y no filtran secretos.
- Las pruebas se pueden ejecutar otra vez con el mismo resultado.

- Las dependencias y versiones necesarias están documentadas.
- El cambio se ha revisado con control de versiones.

8. Laboratorio práctico

Resuelve los ejercicios sin copiar el código anterior. Guarda cada solución en un archivo separado y añade comentarios solo cuando aclaren una decisión. Si trabajas en pareja, una persona implementa y la otra revisa entradas, salidas y casos límite; después intercambiad los papeles.

Práctica A · Básica

Crea un ejemplo mínimo que demuestre coroutines. Incluye una entrada válida, un resultado esperado y una explicación de tres o cuatro líneas sobre cómo funciona.

Práctica B · Intermedia

Extiende el ejemplo con una segunda condición de negocio relacionada con suspend, launch, async, scopes y cancelación. Separa la lógica repetible y crea al menos tres casos de prueba: normal, límite y dato inválido.

Práctica C · Aplicación DAM/DAW

Integra la solución en una pequeña aplicación de consola o en un módulo de tu proyecto. Define una interfaz clara, valida las entradas en el límite adecuado y evita mezclar el acceso a datos con la lógica central.

Práctica D · Revisión

Entrega el código a otra persona sin explicarlo oralmente. Pídele que identifique qué hace, qué puede fallar y cómo probarlo. Corrige cualquier parte que dependa de una explicación que solo tú conoces.

Criterios de evaluación

- Cumple el enunciado y produce resultados verificables.
- Maneja al menos un caso límite relevante.
- Tiene nombres claros y estructura sencilla.
- Incluye pruebas o pasos de reproducción.
- Explica una decisión técnica y una limitación.

9. Soluciones orientativas y autoevaluación

No existe una única solución válida. La revisión debe comprobar el contrato, no si el código se parece exactamente al de otra persona. Para cada práctica, compara las entradas, el resultado y la estrategia de validación.

Solución orientativa

Una solución aceptable para «Coroutines» identifica los datos relevantes, aplica la regla principal de forma explícita y produce una salida que se puede comprobar. Si el caso admite valores inválidos, los rechaza o los representa mediante un error claro. La lógica repetida se extrae solo cuando mejora la comprensión o evita inconsistencias.

Rúbrica de revisión

Criterio	0 · Por revisar	1 · Adecuado	2 · Sólido
Corrección	No cumple el contrato	Cumple el caso básico	Incluye límites y errores
Claridad	Difícil de seguir	Legible en general	Nombres y estructura expresivos
Pruebas	Sin comprobaciones	Un caso válido	Normal, límite y error
Justificación	Sin explicación	Explica la idea	Compara alternativas

Preguntas para explicar en voz alta

- ¿Qué problema resuelve coroutines y cuándo no lo utilizarías?
- ¿Qué prueba añadirías primero si apareciera un fallo?
- ¿Qué parte cambiarías si el requisito creciera?
- ¿Qué información consultarías en la documentación oficial?

10. Repaso, glosario y fuentes

Resumen

Has estudiado coroutines desde el concepto, el diseño, la implementación y las pruebas. Antes de continuar, intenta explicar el tema sin leer y reconstruye el ejemplo desde cero. Si no puedes explicar un paso, vuelve a la sección correspondiente y crea una prueba pequeña.

Glosario

Coroutines: concepto central del capítulo. Contrato: condiciones de entrada y garantías de salida. Caso límite: frontera o supuesto poco común. Regresión: fallo que reaparece tras un cambio. Documentación primaria: referencia mantenida por el equipo responsable de la tecnología.

Fuentes oficiales de consulta

Kotlin Documentation · Kotlin Standard Library · Kotlin Coroutines Guide · Kotlin Documentation: Coroutines basics